SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA5116 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

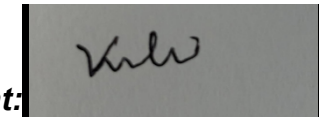
Assessment Tool Weightage: 20%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
CASE STUDY

Code of Conduct:

I _____VARSHA.S/192511358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

1. Weak Password Hashing in Web Application
A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.
Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System
A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.
Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System
A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.
Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System
A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.
Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage
A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices. Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

- **1. Weak Password Hashing in Web Application**

Answer

Introduction

Password hashing is used to securely store user passwords in databases. A hash function converts a password into a fixed-length value that cannot easily be reversed. However, using **unsalted SHA-1** makes passwords vulnerable to attacks.

Analysis of the Vulnerability

Lack of Salting

- A salt is a unique random value added to each password before hashing.
- Without salting:
  - Identical passwords generate identical hashes.
  - Attackers can use precomputed rainbow tables.
  - Multiple user passwords can be cracked quickly.

Use of SHA-1

- SHA-1 is an outdated cryptographic hash algorithm.
- It is designed to be fast, making brute-force attacks easier.
- It has known collision vulnerabilities.
- It is no longer recommended for password storage.

Secure Password Hashing Strategy

Use modern password hashing algorithms such as:

- **Argon2** (recommended)
- **bcrypt**
- **scrypt**

These algorithms:

- Automatically generate unique salts.
- Are computationally expensive.
- Resist brute-force and GPU attacks.
- Increase attacker cost significantly.

Additional Security Measures

- Use long, random salts.

- Apply multi-factor authentication (MFA).
- Enforce strong password policies.
- Rate-limit login attempts.
- Monitor suspicious login activities.

Justification

Argon2 provides better resistance against brute-force and hardware-based attacks because it is memory-intensive and slow, making password cracking extremely difficult.

Conclusion

Unsalted SHA-1 exposes passwords to rainbow table and brute-force attacks. Modern password hashing algorithms like Argon2 with unique salts provide strong protection and significantly improve password security.

- **2. Digital Signature Compromise in Financial System**

Answer

Introduction

Digital signatures ensure authentication, integrity, and non-repudiation in financial transactions. If a user's private key is compromised, attackers can generate valid signatures.

Impact on Authentication

- The attacker appears as the legitimate user.
- Unauthorized transactions become accepted.
- Identity verification fails.

Impact on Non-Repudiation

- Genuine users cannot prove they did not authorize transactions.
- The system cannot distinguish legitimate signatures from forged ones.
- Trust in the digital signature system decreases.

Mitigation Strategies

1. Hardware Security Modules (HSMs)

- Store private keys securely.
- Prevent key extraction.

2. Multi-Factor Authentication

- Require OTP or biometric verification.
- Adds an additional security layer.

3. Key Rotation

- Replace cryptographic keys periodically.
- Limits damage from compromised keys.

4. Certificate Revocation

- Revoke compromised certificates immediately.
- Issue new certificates.

5. Secure Key Storage

- Encrypt private keys.
- Protect with strong passwords.

6. Continuous Monitoring

- Detect unusual transactions.
- Generate alerts for suspicious activities.

Conclusion

Private key compromise breaks authentication and non-repudiation. Strong key management, HSMs, MFA, certificate revocation, and continuous monitoring effectively reduce such risks.

- **3. Kerberos Authentication Failure in Distributed System**

Answer

Introduction

Kerberos is a network authentication protocol that uses tickets and timestamps to authenticate users securely.

Impact of Time Synchronization

Kerberos relies heavily on synchronized system clocks.

If clocks differ:

- Tickets appear expired.
- Tickets may appear invalid.
- Authentication requests fail.
- Users cannot access network resources.

Large clock differences may also:

- Increase replay attack risks.
- Cause denial of service.

Solutions

1. Network Time Protocol (NTP)

- Synchronize all servers with a trusted time server.
- Maintain accurate clocks.

2. Configure Clock Skew

- Allow a small time difference (typically 5 minutes).

3. Redundant Time Servers

- Use multiple synchronized NTP servers.
- Improve availability.

4. Continuous Monitoring

- Monitor time synchronization.
- Alert administrators when clocks drift.

5. Secure Time Sources

- Protect NTP servers against attacks.

Benefits

- Reliable authentication
- Reduced ticket rejection
- Better security
- Improved user experience

Conclusion

Kerberos authentication depends on synchronized clocks. Using NTP, clock skew configuration, redundant time servers, and monitoring ensures reliable authentication and system security.

- **4. Certificate Authority (CA) Breach in PKI System**

Answer

Introduction

A Certificate Authority (CA) issues trusted X.509 digital certificates. If a CA is compromised, attackers can issue fake certificates.

Impact of the Breach

Loss of Trust

- Fake certificates appear legitimate.
- Users trust malicious websites.

Man-in-the-Middle (MITM) Attacks

- Attackers intercept encrypted communication.
- Sensitive information can be stolen.

Identity Impersonation

- Fake certificates allow attackers to impersonate trusted domains.

Compromised Secure Communication

- HTTPS security becomes unreliable.

Detection Mechanisms

- Certificate Transparency Logs
- Certificate monitoring
- Browser certificate warnings
- Certificate pinning

Mitigation Strategies

1. Immediate Certificate Revocation

- Revoke compromised certificates.

2. Issue New Certificates

- Generate new keys and certificates.

3. Multi-Level CA Security

- Protect root CA with Hardware Security Modules.

4. Certificate Transparency

- Publish certificates in public logs.

5. Regular Security Audits

- Monitor CA infrastructure.

6. Short Certificate Validity

- Reduce certificate lifetime.

Conclusion

A CA breach destroys trust in PKI systems. Certificate transparency, revocation, HSMs, certificate pinning, and continuous monitoring help restore trust and protect secure communications.

- **5. Integrity Verification in Cloud Data Storage**

Answer

Introduction

Hashing ensures that stored cloud data has not been modified. Improper implementation of hashing weakens integrity verification.

How Incorrect Hash Validation Causes Integrity Failure

Improper implementation includes:

- Comparing incorrect hashes.
- Using weak hash algorithms.
- Not verifying hashes after download.
- Failing to update stored hashes.
- Accepting corrupted hashes.

As a result:SSSS

- Modified files remain undetected.
- Attackers replace files.
- Users receive corrupted or malicious data.

Robust Integrity Verification Mechanism

1. SHA-256 or SHA-3

- Use strong cryptographic hash algorithms.

2. Digital Signatures

- Verify both integrity and authenticity.

3. Hash Verification During Upload and Download

- Compare newly generated hashes with stored values.

4. Message Authentication Codes (HMAC)

- Protect against unauthorized modification.

5. Merkle Trees

- Efficiently verify integrity of large datasets.

6. Regular Integrity Audits

- Periodically verify stored data.

Justification

Using SHA-256 together with HMAC or digital signatures ensures that any modification to cloud data is immediately detected. Merkle trees provide efficient verification for large-scale cloud storage systems.

Conclusion

Incorrect hash validation leads to integrity failures and undetected data tampering. Strong hash algorithms, HMAC, digital signatures, Merkle trees, and periodic integrity checks provide reliable protection against data modification.

These answers are based on the five case-study questions in your uploaded assessment.